



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(VI)

分布式机器学习的优化

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

寻找最佳训练配置

步骤1：在内存中拟合一个训练步骤

首先，需要弄清楚如何在 GPU 上安装一个完整的模型实例。有两种常见情况

1. GPU丰富的资源- 当有大量可用的GPU时：

- ✓ 对于参数低于 10B 的模型，可以使用单一的并行技术，例如张量并行或 ZeRO-3/DP，并在 8 块 GPU 上进行全面重新计算。
- ✓ 对于参数介于 10B 到 100B 之间、需要超过 8 个 GPU 的型号，有几个选择：
 - ✓ 将张量并行性 (TP=8) 与流水线并行性结合
 - ✓ 将张量并行性 (TP=8) 与数据并行性 (ZeRO-3) 结合
 - ✓ 仅使用 ZeRO-3 (即纯数据并行)

寻找最佳训练配置

步骤1：在内存中拟合一个训练步骤

首先，需要弄清楚如何在 GPU 上安装一个完整的模型实例。有两种常见情况

1. GPU丰富的资源 🤖 - 当有大量可用的GPU时：

- ✓ 在 512+张GPU 规模下，纯数据并行/ZeRO-3 由于通信成本会开始变得低效——不如将数据并行与张量或流水线并行结合使用。
- ✓ 在 1024+ 张GPU 规模下，推荐的设置可能是张量并行（TP=8）配合数据并行（ZeRO-2）和流水线并行。

注意事项

- 对于非常长的序列，可能需要在节点间添加上下文并行。
- 对于专家混合架构，使用跨节点的专家并行性会更有优势。

寻找最佳训练配置

步骤1：在内存中拟合一个训练步骤

首先，需要弄清楚如何在 GPU 上安装一个完整的模型实例。有两种常见情况

2. GPU 资源不足 🤖 的情况——当可能 GPU 资源不足时：

- ✓ 可以启用完全激活的重新计算，用部分计算换取内存（同时训练速度稍微慢一点）。
- ✓ 可以增加梯度累积，以便在有限内存下处理更大批量。

注意事项

- 对于非常长的序列，可能需要在节点间添加上下文并行。
- 对于专家混合架构，使用跨节点的专家并行性会更有优势。

寻找最佳训练配置

步骤2：实现目标的全局批次规模

根据第一步在微批量和 DP 方面的表现，当前的批量可能太小或太大。现在是达到目标批次规模的时候了。

为了增加当前的全局批次规模：

- ✓ 可以扩大数据并行性或梯度累积步骤。
- ✓ 对于长序列，可以利用上下文并行性。

为了减少当前的全局批次规模：

- ✓ 可以减少数据并行性，转而采用其他并行化策略。
- ✓ 对于长序列，可以减少上下文并行性。

寻找最佳训练配置

第三步：优化训练产出

希望确保训练运行尽可能快，以便所有宝贵的 GPU 始终被充分利用。只要内存和通信不成为瓶颈，可以尝试以下方法：

- ✓ 扩大张量并行度（利用快速节点内带宽），直到接近节点大小，以便可以减少其他形式的并行度。
- ✓ 在保持目标批量大小的同时，使用ZeRO-3增加数据并行度。
- ✓ 当数据并行度通信开始成为瓶颈时，过渡到使用流水线并行度。
- ✓ 逐一尝试扩大不同的并行度。
- ✓ 对微批大小(*mbs*)进行实验，以实现最大全局批大小、模型大小、计算和通信之间的最佳平衡。

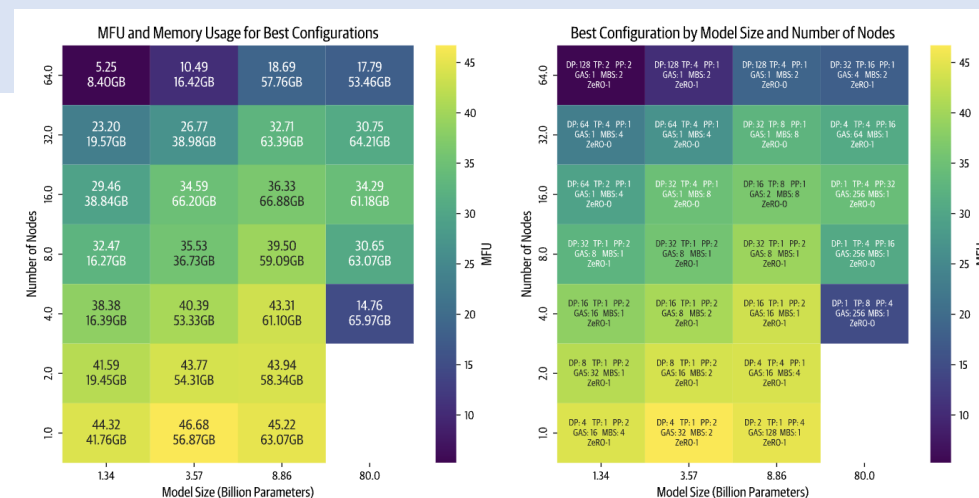
对数千个配置进行基准测试

第三步：优化训练产出

现在已经详细介绍了步骤，让将这个搜索过程应用到实际应用中。

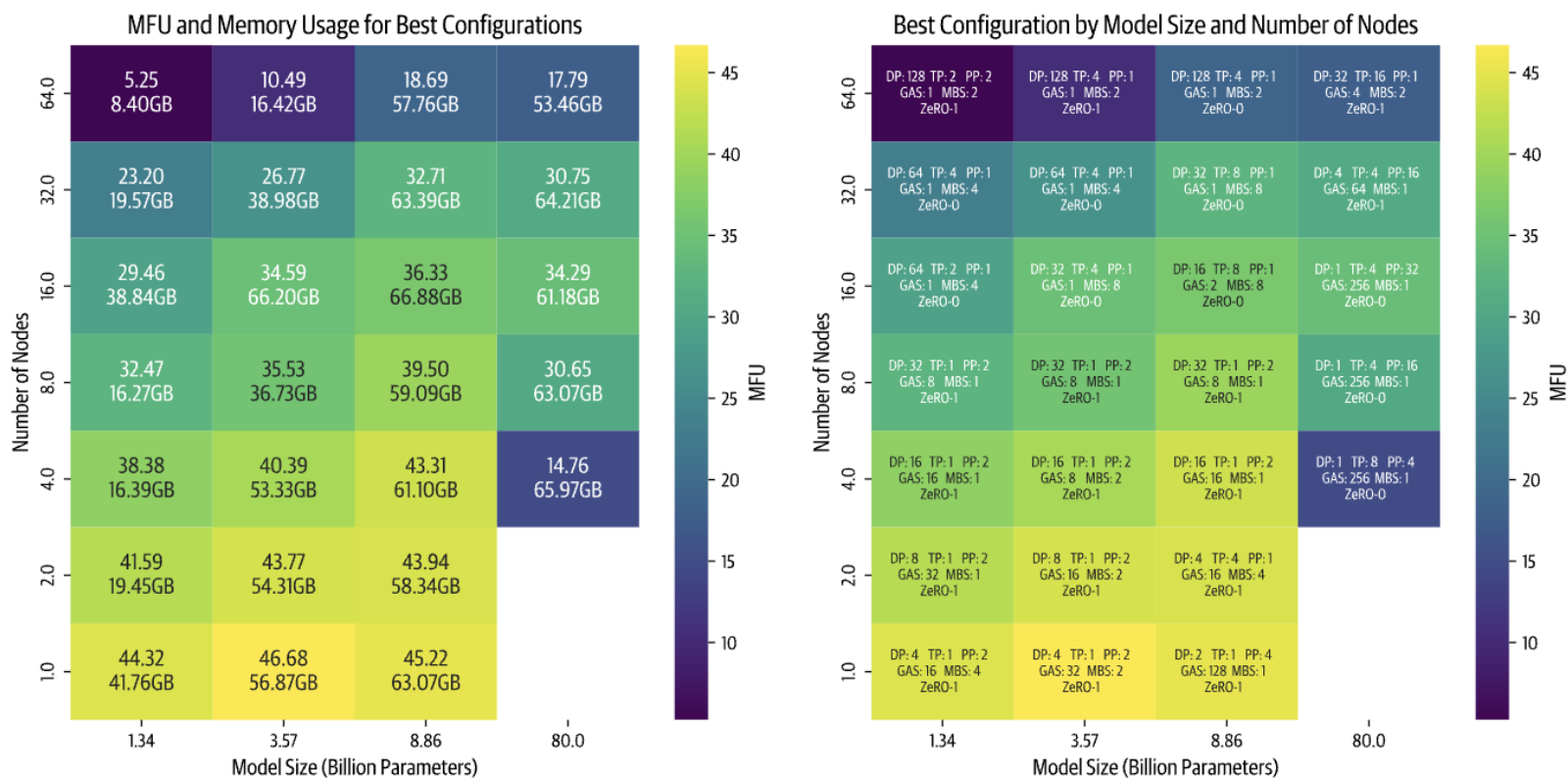
在数千个分布式配置上基准测试，涵盖了很多模型大小，以及一个非常大的集群配置数量（即，8xH100s的1-64节点）。

所有以下基准测试都是在序列长度为4,096和全局批量大小为1M个标记的情况下进行的。收集每个模型和集群大小的最佳配置，并在以下热图中进行了绘制：



对数千个配置进行基准测试

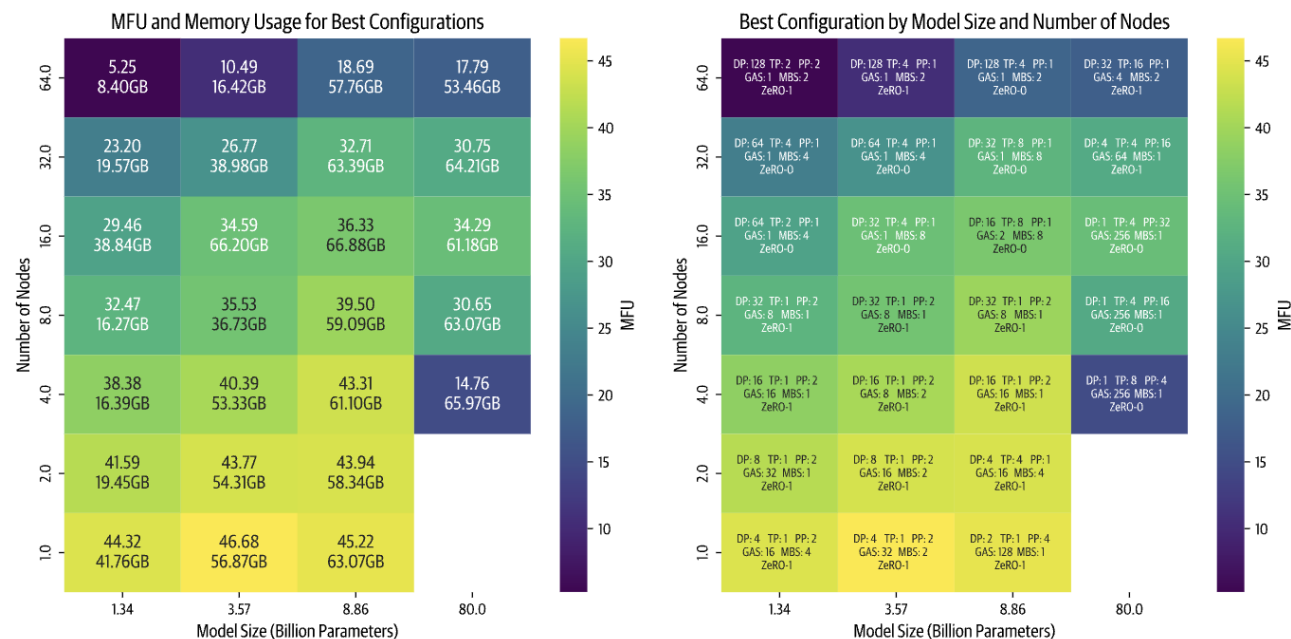
热力图可视化展示了不同模型大小和计算节点数量（每个节点有8个GPU）下的最佳训练配置。对于每种组合，配置细节包括数据并行（DP）、张量并行（TP）、流水线并行（PP）、梯度累积步骤（GAS）、微批大小（MBS）和ZeRO优化阶段。颜色强度表示模型FLOPs利用率（MFU），颜色越亮代表效率越高。



对数千个配置进行基准测试

从这一可视化中，可以得出几个重要的结论

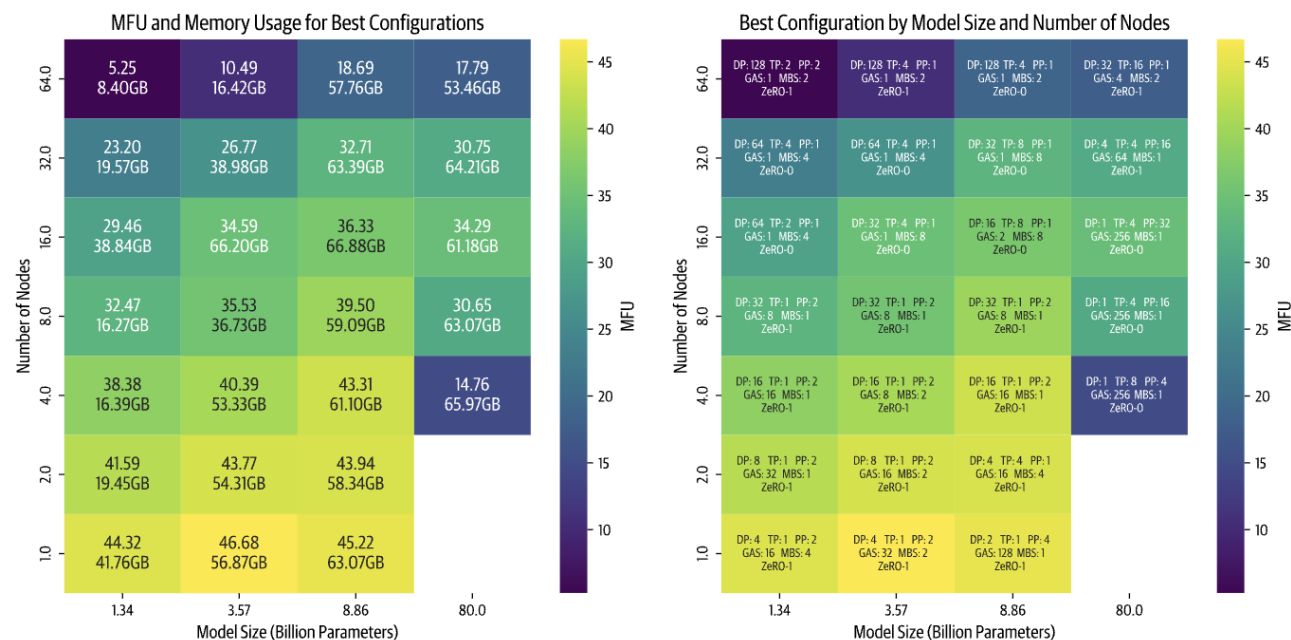
首先，随着节点数量的增加（更高的并行性），观察到效率下降。这种影响在较小的模型中尤为明显，这些模型的计算与模型大小之比较低。虽然可能会通过增加批量大小来补偿小模型的大小，但受限于1M的全局批量大小限制。



对数千个配置进行基准测试

从这一可视化中，可以得出几个重要的见解

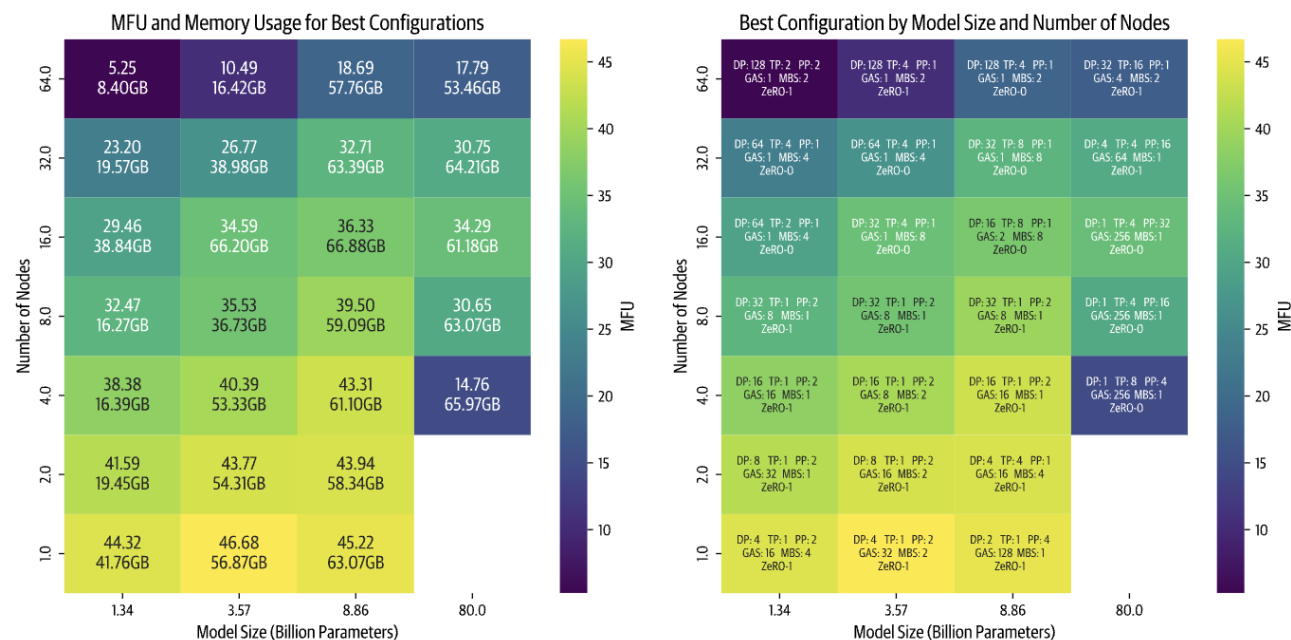
其次，较大的模型提出了不同的挑战。随着模型大小的增加，内存需求大幅增长。这导致在节点较少的情况下出现两种情况：要么模型根本无法适应，要么由于接近GPU内存限制而运行效率低下（例如，在4个节点上对80B参数模型进行训练）。



对数千个配置进行基准测试

从这一可视化中，可以得出几个重要的见解

最后，基准测试显示了性能在很大程度上取决于实现质量。当首次实现这两种并行策略时，张量并行性优于流水线并行性。经过优化PP代码后，它成为了更快的选项。



深入GPU——融合、线程和混合

截至目前

- ✓ 的讨论主要集中在模型操作的较高层次组织上。在各种加速器上移动计算，考虑到一般的内存限制和计算单元的高级调度。
- ✓ 但这些都忽略了可以通过仔细理解的模型操作在每个GPU上的调度和执行来在更低层次进行的所有优化。

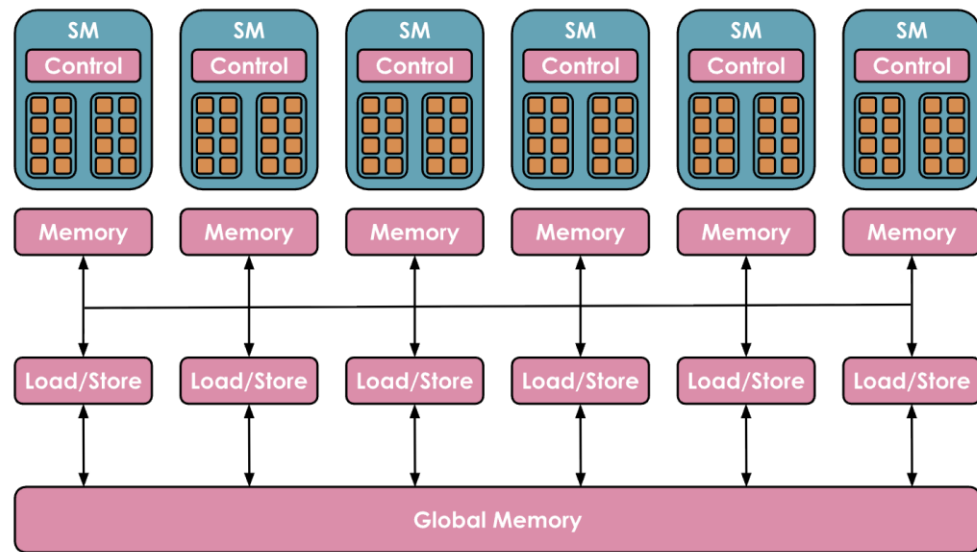
深入探讨GPU架构的细节

- ✓ 将特别关注NVIDIA的GPU架构，但正如通常情况，这些一般性思想可以在类似的加速器单元上重用。
- ✓ 在介绍FlashAttention、如何在GPU上高效调度工作负载以及如何在GPU上高效使用各种精度之前，将简要解释GPU的组织方式。

深入GPU——融合、线程和混合

GPU入门

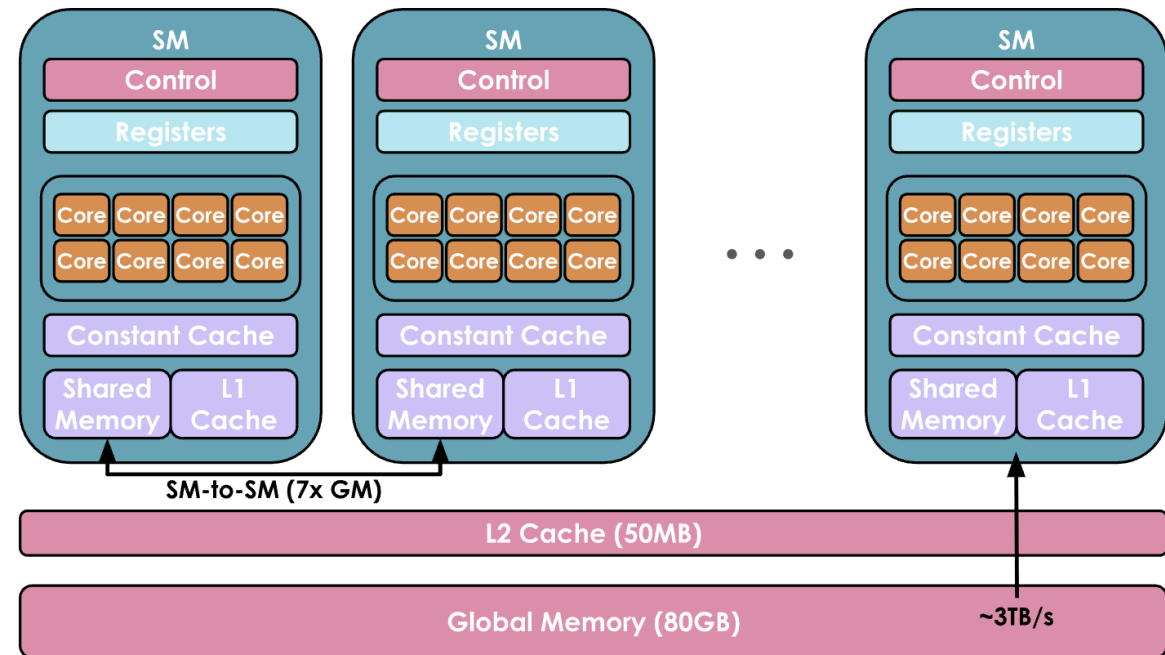
- ✓ 通常，GPU具有非常分层的组织结构。在计算方面，GPU由称为流式多处理器（SMs）的计算单元数组组成。每个SM包含并控制一组流式处理器，也称为核心。
- ✓ 例如，NVIDIA H100 GPU有132个SM，每个SM有128个核心，总共达到16,896个核心（详情请参阅文档中的张量核心），每个核心都能同时处理多个线程。



深入GPU——融合、线程和混合

GPU入门

- ✓ 内存侧也是高度分层的，有多个缓存和内存层级。寄存器是最小的单元，在执行期间属于线程私有。共享内存和L1缓存是在单个SM上运行的线程之间共享的。
- ✓ 更高一层是所有SM共享的L2缓存，最后是全局内存，这是GPU上最大的内存（例如，H100宣传的80GB），但也是访问和查询速度最慢的。



深入GPU——融合、线程和混合

GPU入门

- ✓ 使用GPU的目标是在可用的核心上尽可能并行地运行尽可能多的工作负载，通过利用计算/内存资源的这种分层组织。
- ✓ 在GPU核心上运行的代码片段称为内核。它可以使用CUDA或Triton等高级语言编写，然后编译成并行线程执行（PTX），这是NVIDIA GPU使用的低级汇编语言。

内核通常按以下方式调度

- ✓ 线程被分组为warp，每个warp包含32个线程。warp中的所有线程同步执行指令，但操作不同的数据部分。
- ✓ warp被分组为更大、更灵活的块（例如，一个块中可能有512或1024个线程），每个块分配给一个单独的SM。一个SM可以并行运行多个块。然而，根据可用资源，并非所有块都能立即分配执行；一些可能被列入等待名单，直到更多资源可用。

深入GPU——融合、线程和混合

通过内核提高性能

- ✓ 如果想添加一个缺少优化内核的新操作或加快现有的PyTorch函数，从头开始编写内核可能看起来是最直接的方法。然而，从头创建高性能的CUDA内核需要丰富的经验，并且学习曲线很陡峭。
- ✓ 通常，更好的开始方式是利用torch.compile，它通过捕获操作并在Triton中生成低级、高性能内核来动态优化PyTorch代码。

假设为指数线性单元（ELU）激活函数编写一个内核：

$$\text{ELU}(x) = \begin{cases} e^x - 1 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

可以首先编写一个简单的PyTorch实现，然后只需添加@torch.compile 之上：

```
@torch.compile
def elu(x, alpha=1.0):
    return torch.where(x < 0, alpha * (torch.exp(x) - 1), x)
```

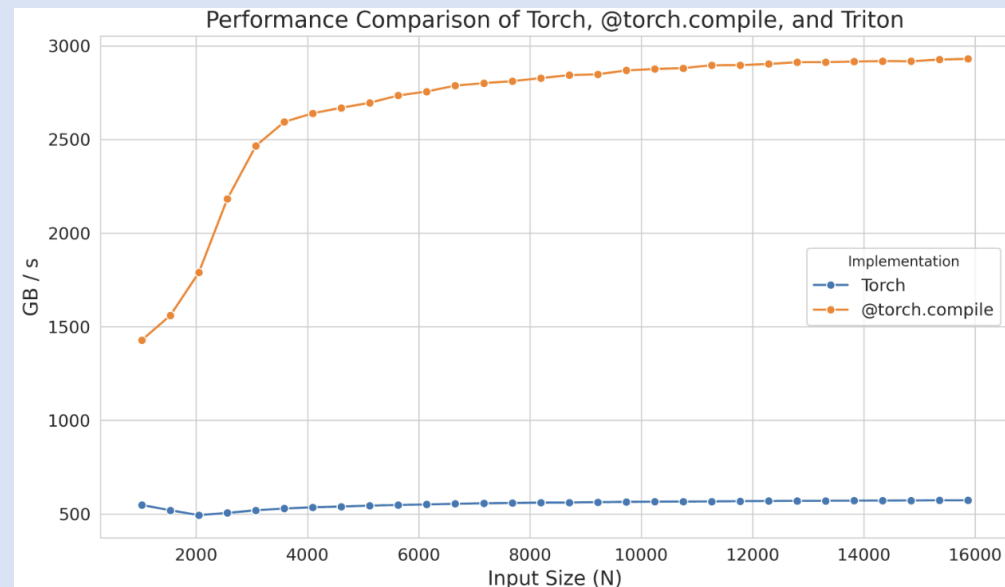

深入GPU——融合、线程和混合

如图所示，编译版和非编译版之间有显著的性能差异，尤其是考虑到只添加了一个装饰器。

然而，如果这种性能提升不足，可以考虑实现 Triton 内核。作为一个起点，查看由 @torch.compile 生成的 Triton 内核。

为此，只需设置环境变量 TORCH_LOGS 为 "output_code"：

```
export TORCH_LOGS="output_code"
```



深入GPU——融合、线程和混合

使用@torch.compile装饰器运行Python脚本

它将生成并输出相应的Triton内核，在这种情况下是：

```
@triton.jit
def triton_(in_ptr0, out_ptr0, xnumel, XBLOCK : tl.constexpr):
    xnumel = 100000000
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[: ]
    xmask = xindex < xnumel
    x0 = xindex
    tmp0 = tl.load(in_ptr0 + (x0), xmask)
    tmp1 = 0.0
    tmp2 = tmp0 < tmp1
    tmp3 = tl_math.exp(tmp0)
    tmp4 = 1.0
    tmp5 = tmp3 - tmp4
    tmp6 = tl.where(tmp2, tmp5, tmp0)
    tl.store(out_ptr0 + (x0), tmp6, xmask)
```

深入GPU——融合、线程和混合

为了提高可读性

可以修改变量名，添加注释，并进行轻微调整（或者请一个LLM为完成），如下所示：

```
@triton.jit
def elu_kernel(input_ptr, output_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
    # Calculate the starting index for this block
    block_start = tl.program_id(0) * BLOCK_SIZE # Create an array of indices for this block
    block_indices = tl.arange(0, BLOCK_SIZE) + block_start # Create an array of indices for this block
    valid_mask = tl.program_id(0) < num_programs # Create a mask to ensure only valid indices are processed
    input_values = tl.load(input_ptr + block_indices, valid_mask) # Load input values based on valid indices
    zero_value = 0.0 # Threshold for ELU activation
    negative_mask = input_values < zero_value exp_values = tl.math.exp(input_values) # Define the ELU output shift
    one_value = 1.0
    shifted_exp_values = exp_values - one_value output_values = tl.where(negative_mask, shifted_exp_values, input_values) # Store the computed output values back to the output pointer
    tl.store(output_ptr + block_indices, output_values, valid_mask)
```

tl.program_id(0)提供了一个唯一的块ID，用它来确定该块将处理哪个数据部分。

深入GPU——融合、线程和混合

为了提高可读性

可以修改变量名，添加注释，并进行轻微调整（或者请一个LLM为完成），如下所示：

```
@triton.jit
def elu_kernel(input_ptr, output_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
    # Calculate the starting index for this block
    block_start = tl.program_id(0) * BLOCK_SIZE # Create an array of indices for this block
    block_indices = block_start + tl.arange(0, BLOCK_SIZE)[:] # Create a mask to ensure only valid indices

    使用这个块ID， block_start计算每个块部分的起始索引，而
    block_indices指定该部分内的索引范围。

    input_values = tl.load(input_ptr + block_indices, valid_mask) # Define the ELU parameters
    zero_value = 0.0 # Threshold for ELU activation
    negative_mask = input_values < zero_value exp_values = tl.math.exp(input_values) # Define the ELU output
    shifted_exp_values = exp_values - one_value output_values = tl.where(negative_mask, shifted_exp_values,
    input_values) # Store the computed output values back to the output pointer
    tl.store(output_ptr + block_indices, output_values, valid_mask)
```

深入GPU——融合、线程和混合

为了提高可读性

可以修改变量名，添加注释，并进行轻微调整（或者请一个LLM为完成），如下所示：

```
@triton.jit
def elu_kernel(input_ptr, output_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
    # Calculate the starting index for this block
    block_start = tl.program_id(0) * BLOCK_SIZE # Create an array of indices for this block
    block_indices = block_start + tl.arange(0, BLOCK_SIZE)[:] # Create a mask to ensure only valid indices
    are processed
    valid_mask = block_indices < num_elements # Load input values from the input pointer based on valid
    indices
    input_values = tl.load(input_ptr + block_indices, valid_mask) # valid_mask确保只处理num_elements内的索引，安全地使用
    zero_value = 0.0 # tl.load加载数据。
    negative_mask = input_values < zero_value exp_values = tl.math.exp(input_values) # Define the ELU parameters
    shift
    one_value = 1.0
    shifted_exp_values = exp_values - one_value output_values = tl.where(negative_mask, shifted_exp_values,
    input_values) # Store the computed output values back to the output pointer
    tl.store(output_ptr + block_indices, output_values, valid_mask)
```

深入GPU——融合、线程和混合

为了提高可读性

可以修改变量名，添加注释，并进行轻微调整（或者请一个LLM为完成），如下所示：

```
@triton.jit
def elu_kernel(input_ptr, output_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
    # Calculate the starting index for this block
    block_start = tl.program_id(0) * BLOCK_SIZE # Create an array of indices for this block
    block_indices = block_start + tl.arange(0, BLOCK_SIZE)[:BLOCK_SIZE] # Create a mask to ensure only valid indices
    # are processed
    valid_mask = block_indices < num_elements # Load input values from the input pointer based on valid
    # indices
    input_values = tl.load(input_ptr + block_indices, valid_mask) # Define the ELU parameters
    zero_value = 0.0 # Threshold for ELU activation
    negative_mask = input_values < zero_value exp_values = tl.math.exp(input_values) # Define the ELU output
    # shift
    one_value = 1.0
    shifted_exp_values = exp_values - one_value output_values = tl.where(negative_mask, shifted_exp_values,
    input_values) # Store the computed output values back to the output pointer
    tl.store(output_ptr + block_indices, output_values, valid_mask)
```

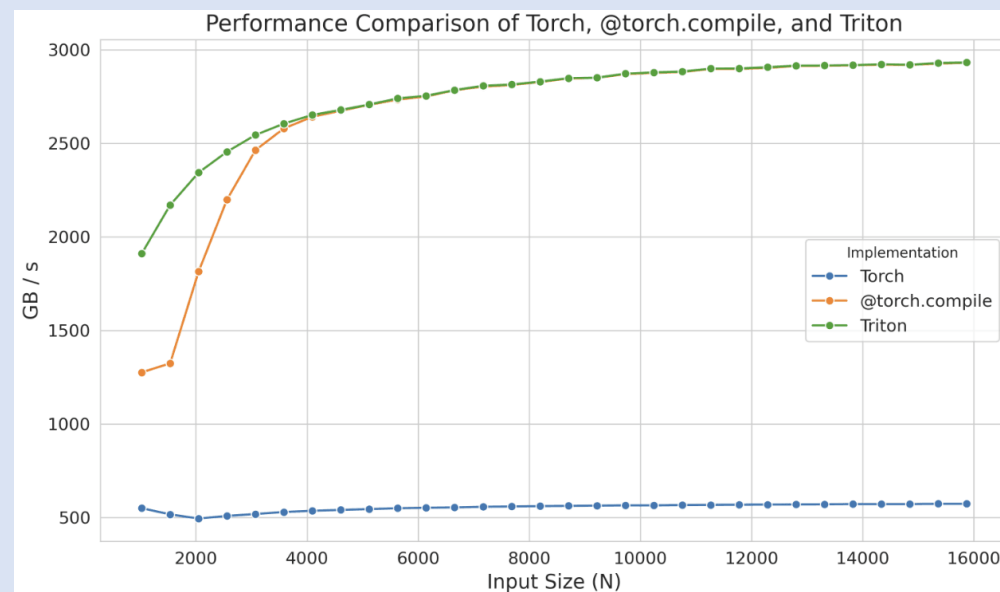
然后应用ELU函数，根据值的正负修改值，并将结果写回内存使用tl.store。

深入GPU——融合、线程和混合

使用 `triton.testing.Benchmark` 对生成的内核进行基准测试

这个独立内核甚至在与 `@torch.compile` 相比尺寸更小的情况下展现出更优越的性能，但这很可能是 `torch.compile` 编译时间的产物。无论如何，与其从头开始，记得可以从这样的生成内核开始，专注于优化它们的性能，这将为节省大量时间。

即使在 Triton 中，有时也无法完全实现设备的峰值性能，因为语言在处理像共享内存和流多处理器内的调度等底层细节方面存在限制。Triton 的功能仅限于块和跨 SM 的块调度。要获得更深入的控制，需要直接在 CUDA 中实现内核，在那里将能够访问所有底层细节。



深入GPU——融合、线程和混合

使用CUDA，可以采用各种技术来提高内核的效率

在这里只介绍几个：优化内存访问模式以减少延迟，使用共享内存来存储频繁访问的数据，以及管理线程工作负载以最小化空闲时间。

总结一下所看到的工具，这些工具让编写内核代码来执行GPU上的指令：

- PyTorch：简单但慢
- `@torch.compile`：简单、快速，但不灵活
- Triton：更难、更快，但更灵活
- CUDA：最难、最快、最灵活（如果用对了）

将从CUDA最频繁使用之一：优化内存访问开始。与缓存相比，GPU中的全局内存具有较长的延迟和较低的带宽，这为许多应用创造了主要的瓶颈。有效地访问全局内存中的数据可以显著提高性能。

深入GPU——融合、线程和混合

内存归约

为了有效地利用全局内存的带宽，理解其架构至关重要。在CUDA设备中，全局内存是通过DRAM实现的。

内存归约利用了DRAM在访问内存地址时以突发方式交付数据的特点

- 每次访问DRAM位置时，一系列连续的位置（包括请求的位置）由DRAM芯片中的多个传感器并行读取。一旦读取，这些数据就可以快速地作为一个突发传输到处理器。
- 在CUDA中，归约利用这种突发行为，通过确保warp中的线程——一组32个线程，它们以锁步的方式执行相同的指令——访问连续的内存位置，从而最大化内存访问效率。
- 例如，如果线程0访问位置M，线程1访问M+1，线程2访问M+2，以此类推，GPU硬件将这些请求归约或合并为一个大型、高效的访问请求，而不是单独处理每个访问。

深入GPU——融合、线程和混合

让以矩阵乘法为例

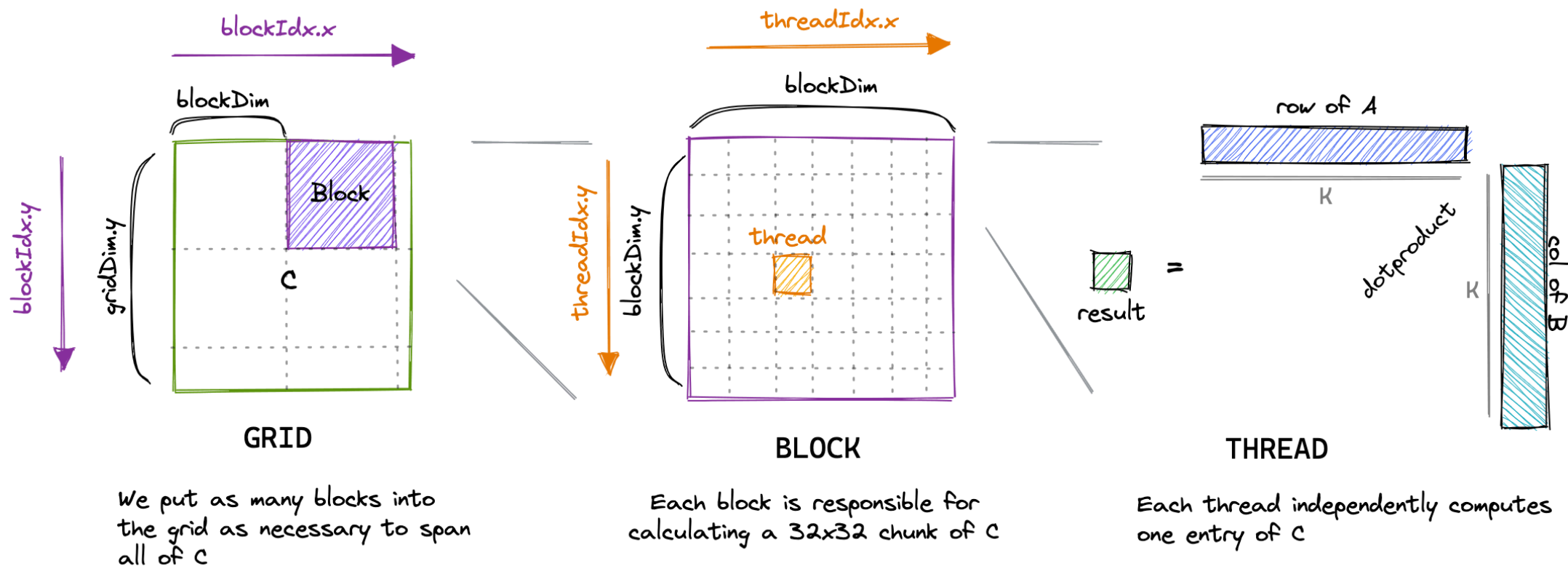
一个简单直接的实施方式是每个线程计算输出矩阵的单个元素，如下所示：

```
__global__ void matmul_naive(int M, int N, int K, const float *A, const float *B, float *C)
{
    const uint x = blockIdx.x * blockDim.x + threadIdx.x;
    const uint y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < M && y < N)
    {
        float tmp = 0.0;
        for (int i = 0; i < K; ++i)
        {
            tmp += A[x * K + i] * B[i * N + y];
        }
        C[x * N + y] = tmp;
    }
}
```

深入GPU——融合、线程和混合

让以矩阵乘法为例

一个简单直接的实施方式是每个线程计算输出矩阵的单个元素，如下所示：



深入GPU——融合、线程和混合

然而，当使用ncu等工具对内核进行剖析时

可以看到问题，包括低内存吞吐量和未联合内存访问：

Memory Workload Analysis				All	
Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.					
Memory Throughput [Gbyte/s]	24.85	Mem Busy [%]		98.40	
L1/TEX Hit Rate [%]	99.13	Max Bandwidth [%]		6.25	
L2 Hit Rate [%]	65.77	Mem Pipes Busy [%]		5.96	

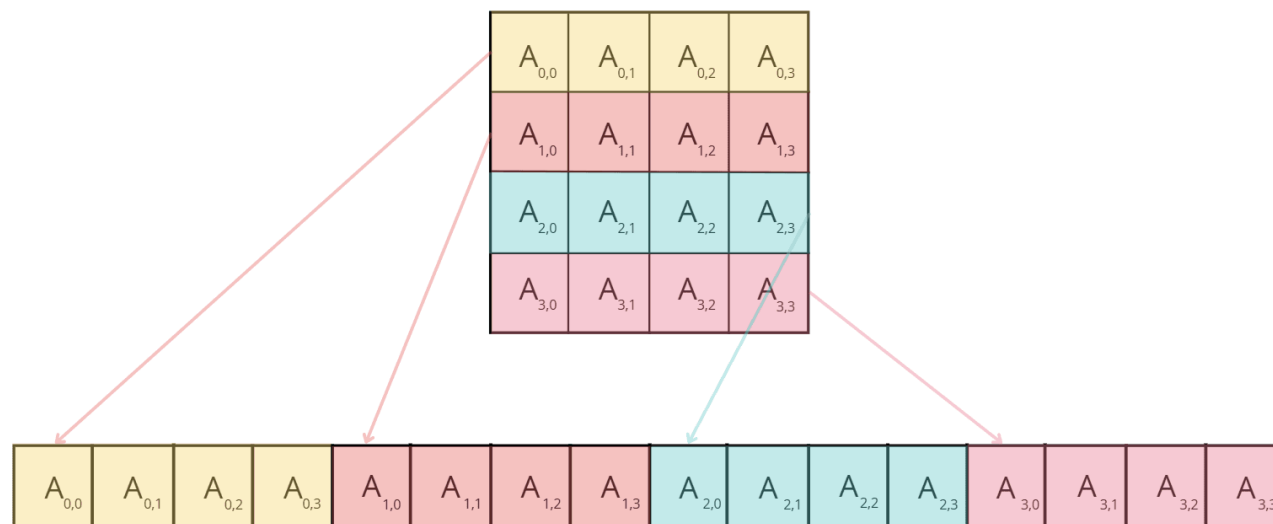
原因在于

- 在这个内核中，具有线程ID（0， 0）和（1， 0）的同一块中的两个线程（最终将位于同一个warp中）将同时从矩阵B的同一列加载，但矩阵A的不同行。

深入GPU——融合、线程和混合

原因在于

- 在这个内核中，具有线程ID $(0, 0)$ 和 $(1, 0)$ 的同一块中的两个线程（最终将位于同一个warp中）将同时从矩阵B的同一列加载，但矩阵A的不同行。
- 由于矩阵元素以行主序存储（意味着行元素在连续的内存地址中），如图所示，线程 $(0, 0)$ 将在第一次迭代 $i = 0$ 时加载 $A_{0,0}$ ，而线程 $(1, 0)$ 将加载 $A_{1,0}$ 。这些元素在内存中并不相邻存储，这种错位将在每次迭代中都存在，从而防止内存访问合并。



深入GPU——融合、线程和混合

为了提高内核的性能

可以将计算坐标x和y的方式更改为以下：

```
const int x = blockIdx.x * BLOCKSIZE + (threadIdx.x / BLOCKSIZE);
const int y = blockIdx.y * BLOCKSIZE + (threadIdx.x % BLOCKSIZE);
if (x < M && y < N)
{
    float tmp = 0.0;
    for (int i = 0; i < K; ++i)
    {
        tmp += A[x * K + i] * B[i * N + y];
    }
    C[x * N + y] = tmp;
}
```

不使用二维块，切换到一维块，并重新定义确定x和y值的方式。在这种新方法中，同一warp内的线程（具有接近的threadIdx.x值）将共享相同的x值，但具有不同的y值。这意味着它们将加载矩阵A的同一行，但矩阵B的不同列。因此，对于行主序矩阵，可以合并内存访问。

深入GPU——融合、线程和混合

对新的内核进行性能分析时

关于未归一化内存访问的警告已经消失， GPU的内存吞吐量大约提高了10倍。

Memory Workload Analysis				All	
Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.					
Memory Throughput [Gbyte/s]	234.57	Mem Busy [%]		57.88	
L1/TEX Hit Rate [%]	94.98	Max Bandwidth [%]		57.87	
L2 Hit Rate [%]	51.51	Mem Pipes Busy [%]		57.87	

现在让来介绍另一种在文献中经常提到的技术：分块。

分块

是一种利用共享内存来优化内存访问模式的技巧。正如之前提到的，GPU上的共享内存是一个小而快的内存区域，可以被一个块内的所有线程访问。

它允许多个线程重用数据，减少从较慢的全局内存中重复加载数据的需求。

例如，在矩阵乘法中，一个块内的每个线程可能需要从两个矩阵中获取元素，比如A和B

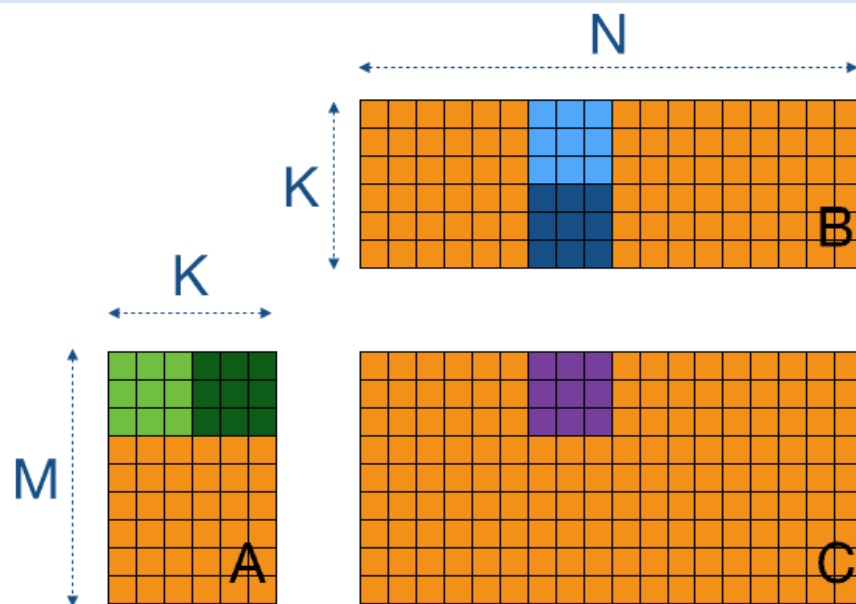
- 如果每个线程独立地从全局内存中加载它需要的行和列，将会有很多冗余加载，因为一个块内的多个线程将访问重叠的数据。
- 相反，可以使用分块技术，将A和B的块一次性加载到共享内存中，使得该块内的所有线程可以重用相同的共享数据。

深入GPU——融合、线程和混合

分块

每个迭代涉及一个块内的所有线程合作加载两个tile——一个来自矩阵A，另一个来自矩阵B——到共享内存中。

具体来说，线程加载一个矩阵A的tile（大小为BLOCK_SIZE_M乘以BLOCK_SIZE_K）和一个矩阵B的tile（大小为BLOCK_SIZE_K乘以BLOCK_SIZE_N）。



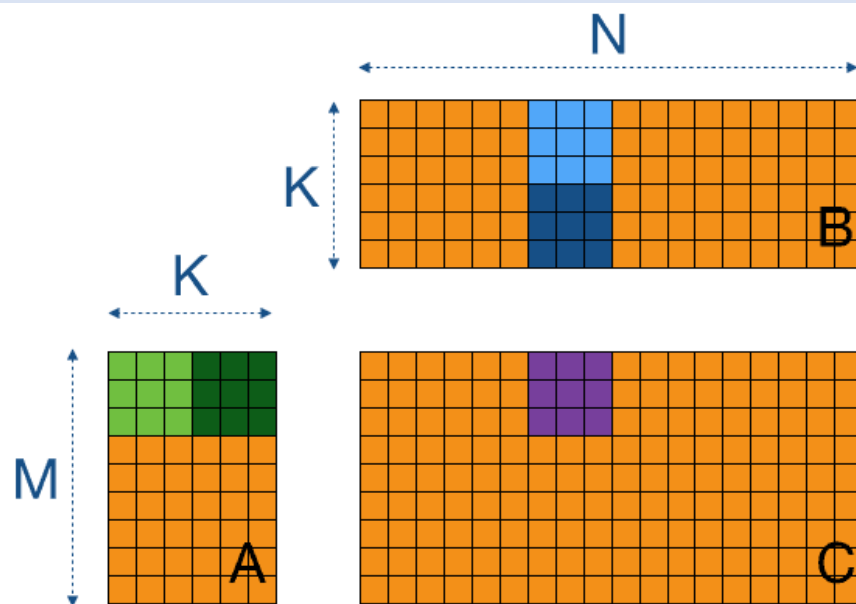
$$\begin{bmatrix} \text{purple tile} \end{bmatrix} = \begin{bmatrix} \text{green tile} \end{bmatrix} \times \begin{bmatrix} \text{blue tile} \end{bmatrix} + \begin{bmatrix} \text{dark green tile} \end{bmatrix} \times \begin{bmatrix} \text{dark blue tile} \end{bmatrix}$$

深入GPU——融合、线程和混合

分块

每个迭代涉及一个块内的所有线程合作加载两个tile——一个来自矩阵A，另一个来自矩阵B——到共享内存中。

一旦块在共享内存中，线程对这些tile执行矩阵乘法，由于所有必要的数据都可以快速访问，因此实现高效计算。tile乘法的结果存储在一个累积矩阵中，该矩阵保存中间结果。在每个迭代之后，当前tile乘法的结果被添加到这个累积矩阵中，直到处理完两个矩阵的所有tile。



$$\begin{bmatrix} \text{purple tile} \end{bmatrix} = \begin{bmatrix} \text{green tile} \end{bmatrix} \times \begin{bmatrix} \text{blue tile} \end{bmatrix} + \begin{bmatrix} \text{dark green tile} \end{bmatrix} \times \begin{bmatrix} \text{dark blue tile} \end{bmatrix}$$

深入GPU——融合、线程和混合

让看看需要理解实现中的重要部分：

```
// Set pointers to the starting elements
A += blockDim * TILE_SIZE * K;
// Start at row = blockDim, column = blockDim
B += blockDim * TILE_SIZE; // Start at row = 0, column = blockDim
C += blockDim * TILE_SIZE * N + blockDim * TILE_SIZE; // Start at row = blockDim, column = blockDim
float sum = 0.0; // The outer loop moves through tiles of A (across columns) and B (down rows)
for (int tileIdx = 0; tileIdx < K; tileIdx += TILE_SIZE) {
    sharedA[localRow * TILE_SIZE + localCol] = A[localRow * K + localCol];
    sharedB[localRow * TILE_SIZE + localCol] = B[localRow * N + localCol]; // Ensure all threads in the
    block have completed data loading
    __syncthreads(); // Shift pointer
    A += TILE_SIZE; B += TILE_SIZE *
    for (int i = 0; i < TILE_SIZE; ++i)
        sum += sharedA[localRow * TILE_SIZE + i] * sharedB[i * TILE_SIZE + localCol];
} // Synchronize again to prevent any thread from loading new data // into shared memory before
others have completed their calculations
__syncthreads();
}
C[localRow * N + localCol] = sum;
```

每个线程首先从矩阵A和矩阵B中各加载一个元素到共享内存中。

在这种情况下，实现内存访问的合并是直接的：通过将threadIdx.x分配为局部索引（localCol），确保同一warp内的线程将访问两个矩阵的相邻元素。

在每个线程将元素加载到共享内存中完成（通过调用__syncthreads()确保）后，它们继续计算两个tile的点积。一旦线程迭代完所有tile——对于A是水平方向，对于B是垂直方向——结果之和存储在矩阵C的相应位置。

深入GPU——融合、线程和混合

线程粗化

分块技术显著提高了内核的性能。然而，在分析 warp 状态时，这些状态量化了在每个状态中花费了多少周期，观察到以下情况：

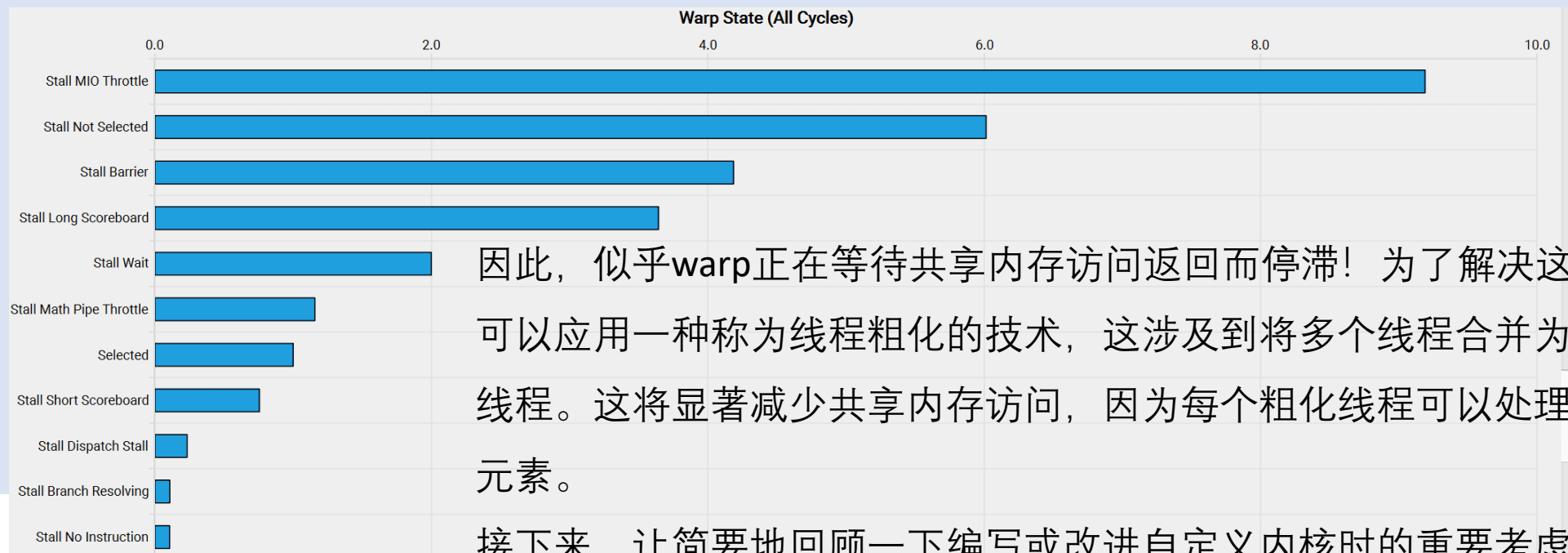


这些神秘状态名称的含义可以在NVIDIA的内核分析指南中找到，在“Warp 停滞原因”部分。在那里，看到 `smsp__pcsamp_warps_issue_stalled_mio_throttle` 表示“Warp停滞等待MIO（内存输入/输出）指令队列不满。这种停滞原因在MIO管道极端利用率的情况下很高，包括特殊数学指令、动态分支以及共享内存指令。当由共享内存访问引起时，尝试使用更少但更宽的加载可以减少管道压力。”

深入GPU——融合、线程和混合

线程粗化

分块技术显著提高了内核的性能。然而，在分析 warp 状态时，这些状态量化了在每个状态中花费了多少周期，观察到以下情况：



因此，似乎warp正在等待共享内存访问返回而停滞！为了解决这个问题，可以应用一种称为线程粗化的技术，这涉及到将多个线程合并为一个粗化线程。这将显著减少共享内存访问，因为每个粗化线程可以处理多个输出元素。

接下来，让简要地回顾一下编写或改进自定义内核时的重要考虑因素的最后一点：最小化控制偏差

最小化控制偏差

流式多处理器是构建用来使用单指令多数据（SIMD）模型执行一个warp中所有线程的。这意味着在任何给定时刻，一条指令被同时提取并执行，用于warp中的所有线程。当一个warp被执行时，其中的线程在数据的不同部分上操作，但遵循相同的指令（因此得名单指令多数据）。

SIMD的主要优势是其效率：负责指令提取和分发的控制硬件在多个执行单元之间共享。这种设计最小化了与控制功能相关的硬件开销，使得更多的硬件可以专注于提高算术吞吐量。

当同一个warp中的线程采取不同的执行路径时，会发生控制偏差。

例如，如果一个条件语句（如if语句）导致一些线程执行一个代码块，而其他线程执行不同的代码块，warp必须序列化这些执行，导致空闲线程等待其他线程完成。为了最小化控制偏差，需要设计内核以确保同一个warp中的线程遵循相同的执行路径。这可以通过重构代码以减少分支、使用确保所有线程遵循相似执行路径的数据结构，或采用预测等技术来实现。

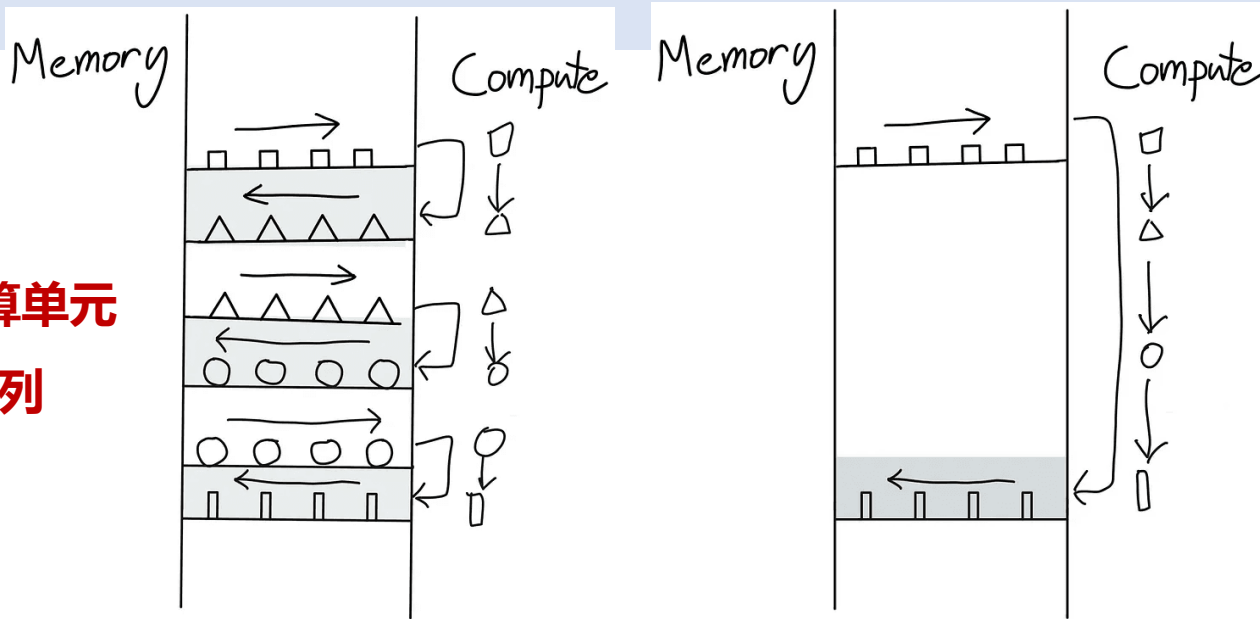
深入GPU——融合、线程和混合

融合内核

现在在几个地方，已经提到了GPU和CPU操作可以是异步的。特别是，CPU上的主机代码可以以非阻塞的方式在GPU上调度工作负载。

这可以用于重叠通信和计算——但它也可以扩展到更一般的概念，即不惜一切代价避免在主机和GPU内核命令之间来回切换。

**需要在全局内存和计算单元
之间来回的内核序列**



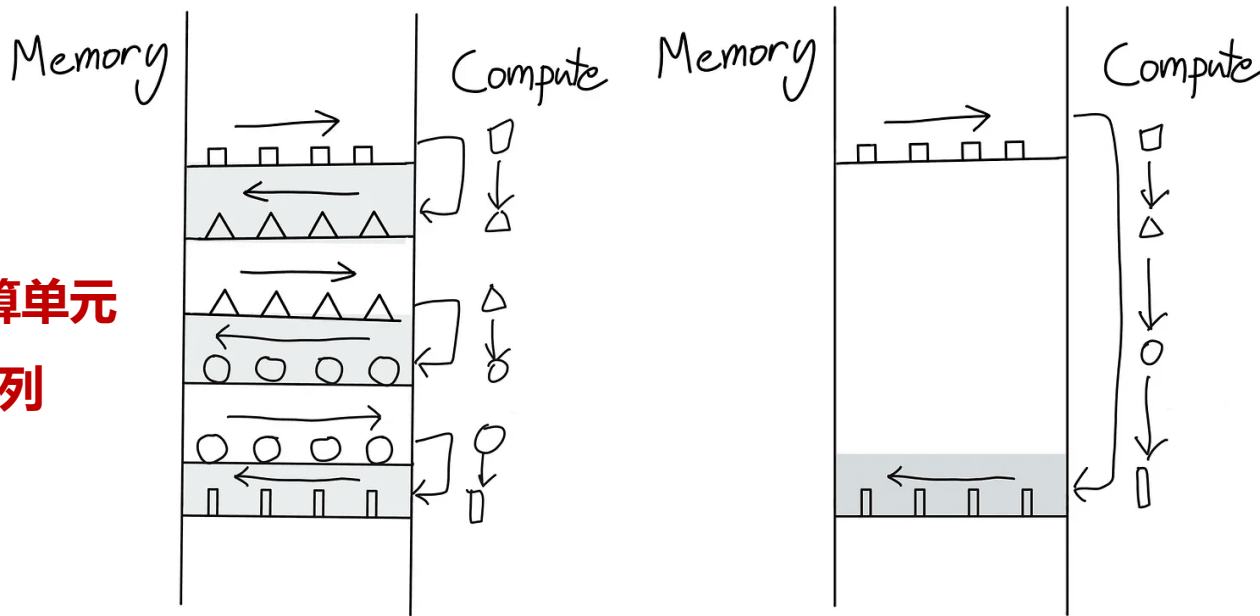
不再把这个三角结构中间结果写回全局内存后再重新读取，而是直接一次性完成所有运算。

深入GPU——融合、线程和混合

如何避免左侧所示的来回操作

最好的办法是尽可能让的GPU尽可能自主。这是通过将尽可能多的连续计算操作打包在一个GPU运行的单一内核中实现的，称为“融合内核”，如右侧所示。

需要在全局内存和计算单元
之间来回的内核序列



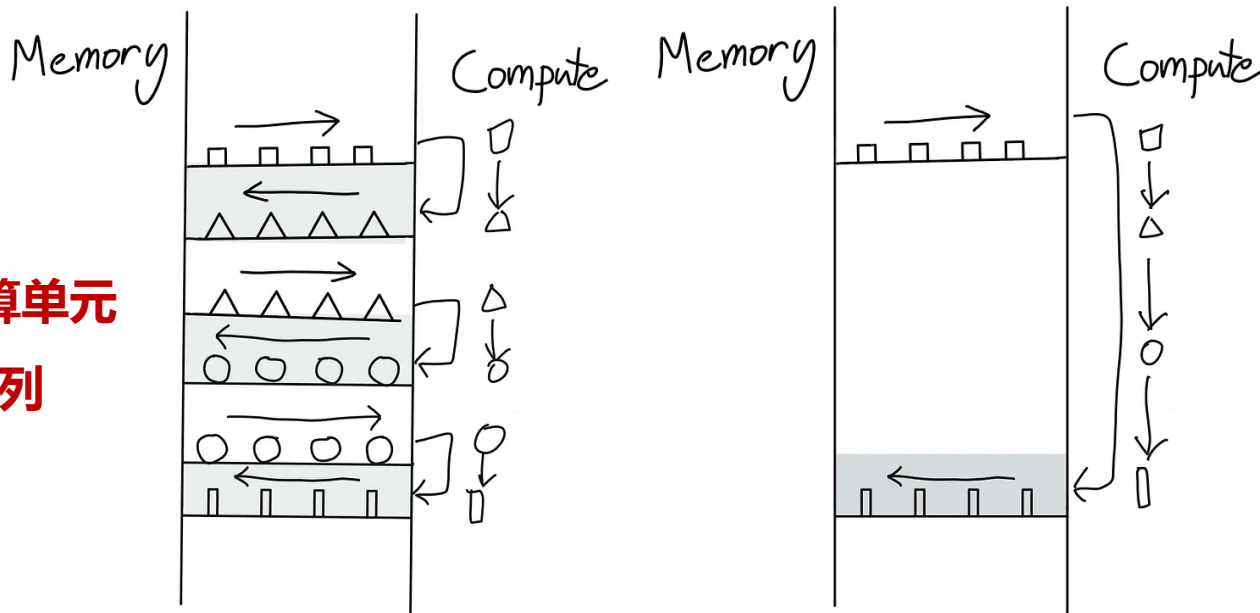
不再把这个三角结构中间结
果写回全局内存后再重新读
取，而是直接一次性完成所
有运算。

深入GPU——融合、线程和混合

如何避免左侧所示的来回操作

融合内核对于独立于彼此在每个输入标记上执行的一系列点状操作特别高效且易于编写。在这种情况下，在将计算值移动到SM内存并启动一个新的内核之前，将它们发送回全局内存是没有意义的。将所有值保留在本地直到所有计算完成要高效得多。

需要在全局内存和计算单元
之间来回的内核序列



不再把这个三角结构中间结
果写回全局内存后再重新读
取，而是直接一次性完成所
有运算。

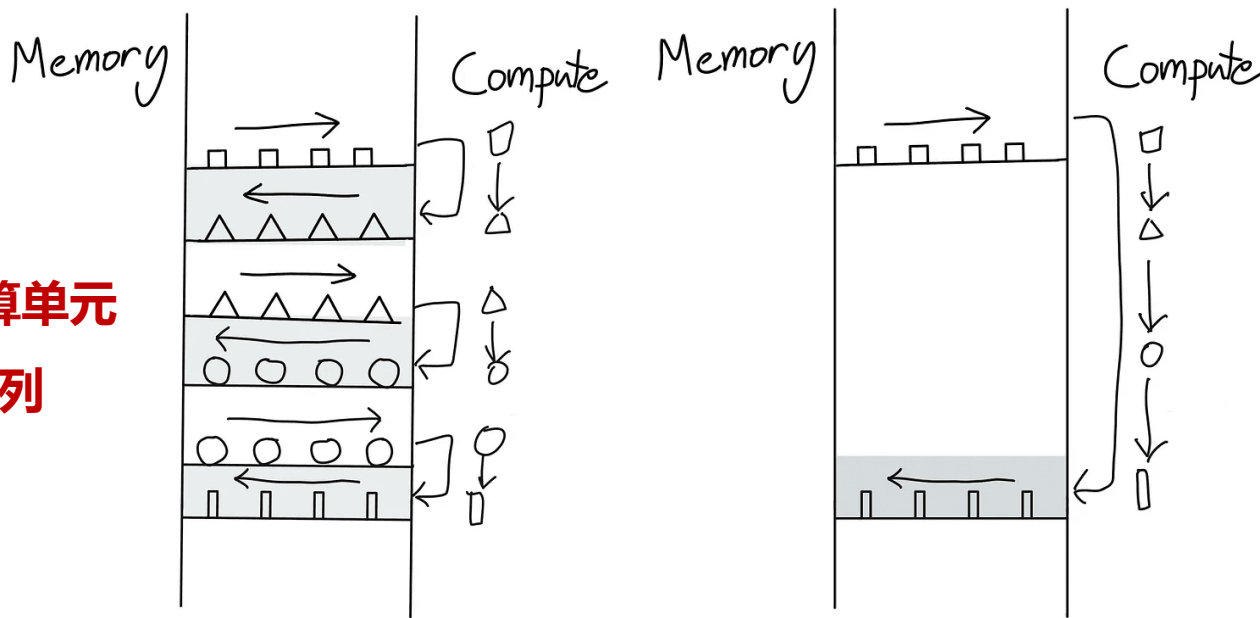
深入GPU——融合、线程和混合

在Transformer模型中

这种“融合”方法可以在每次有连续的点状操作时应用，例如在涉及LayerNorm层的计算中。

现在已经拥有了欣赏真正的内核工程杰作——FlashAttention所必需的所有理解。

需要在全局内存和计算单元
之间来回的内核序列



不再把这个三角结构中间结果写回全局内存后再重新读取，而是直接一次性完成所有运算。

深入GPU——融合、线程和混合

FlashAttention

FlashAttention是由Tri Dao提出的，建议通过编写定制的CUDA内核来优化注意力计算，使它们更快、更节省内存。FlashAttention背后的想法是高效利用GPU的各种内存，避免过度依赖最慢的那个：全局内存。

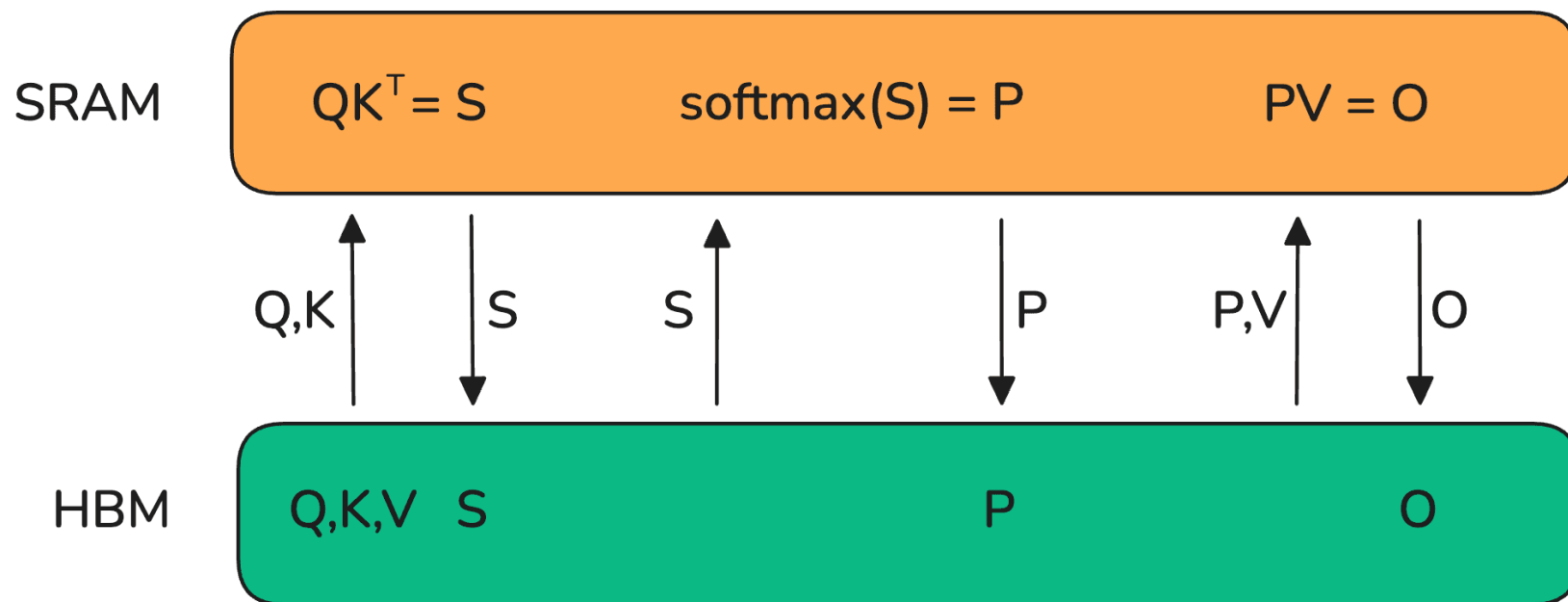
需要在全局内存和计算单元之间来回的内核序列

- ✓ 现代GPU中的全局内存通常使用一种称为高带宽内存（HBM）的技术，尽管它的名字叫“高带宽”，但在GPU内存层次结构中，它的速度比SRAM慢。
- ✓ 当讨论FlashAttention实现细节时，这个HBM术语将非常重要。

深入GPU——融合、线程和混合

注意力机制的基本实现涉及大量在内存和工作节点之间的传输

它需要在HBM中实现S矩阵（其中 $S = QK^T$ ，注意力分数）和P矩阵（其中 $P = \text{softmax}(S)$ ，归一化注意力权重），这意味着结果需要发送到HBM，然后再返回SRAM进行下一次计算：



深入GPU——融合、线程和混合

由于HBM的带宽要低得多，这为注意力计算引入了严重的瓶颈

- 关键
- 避免
- 因此
- 在这
- 下文

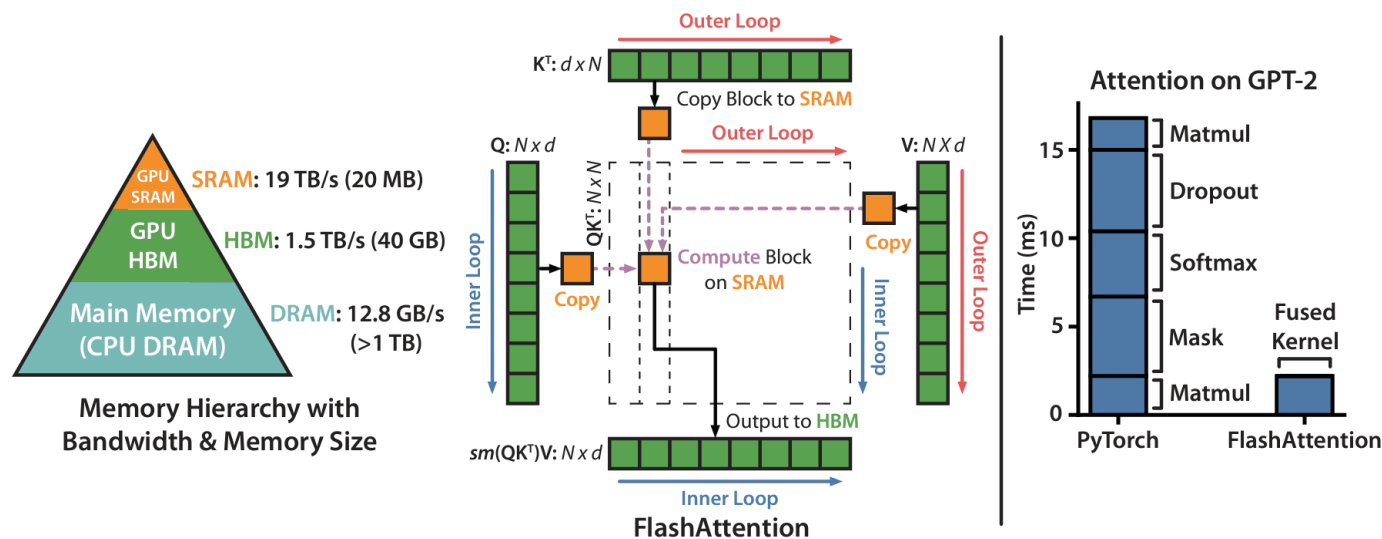


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the $\mathbf{K}$ and $\mathbf{V}$ matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of $\mathbf{Q}$ matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6 $\times$ speedup on the attention computation.

做得更好，完全

io

之一（在长上

FlashAttention解决了模型训练中的许多瓶颈，迅速成为所有Transformer执行注意力的默认方式

- 通过避免实现S矩阵，减少了注意力的内存负担。
- 还消除了注意力中 $O(S^2)$ 成本的大部分原始影响。
- 所有线性注意力及其近似注意力的次二次方法（在Transformer架构发明后不久开发）都已被弃用，转而采用这种精确且快速的FlashAttention实现和机制。

FlashAttention-2和-3

与FlashAttention-1相比，FlashAttention-2和-3的改进更多是关于对其低级实现进行更具体的调整，以适应GPU，具体包括：

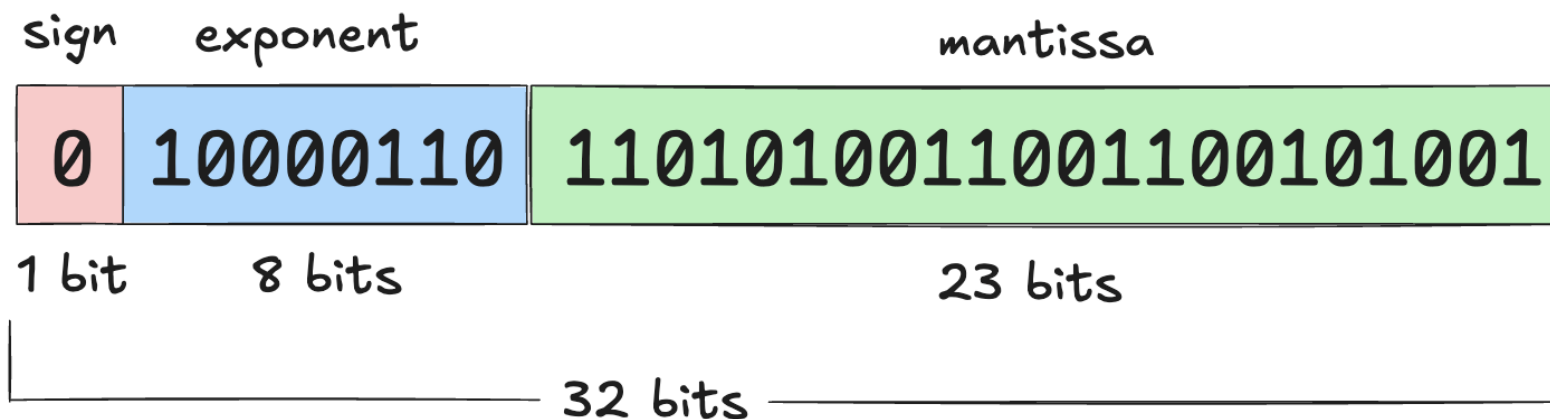
- （1）尽可能减少非matmul操作的数量，
- （2）仔细分配wrap和线程块之间的工作负载（对于FlashAttention-2），以及
- （3）针对最新的Hopper（H100）架构上的FP8和Tensor Core支持进行优化（对于FlashAttention-3）。

深入GPU——融合、线程和混合

混合精度训练

混合精度训练涉及在训练时混合不同的精度。PyTorch张量的默认数值精度是单精度浮点格式，也称为FP32或float32，这意味着每个存储的数字占用32位，或4字节。表示数字的可用位被分为三个部分：

- 符号：第一个位确定数字是正数还是负数
- 指数：控制数字的大小
- 尾数：确定数字的有效数字



深入GPU——融合、线程和混合

混合精度训练

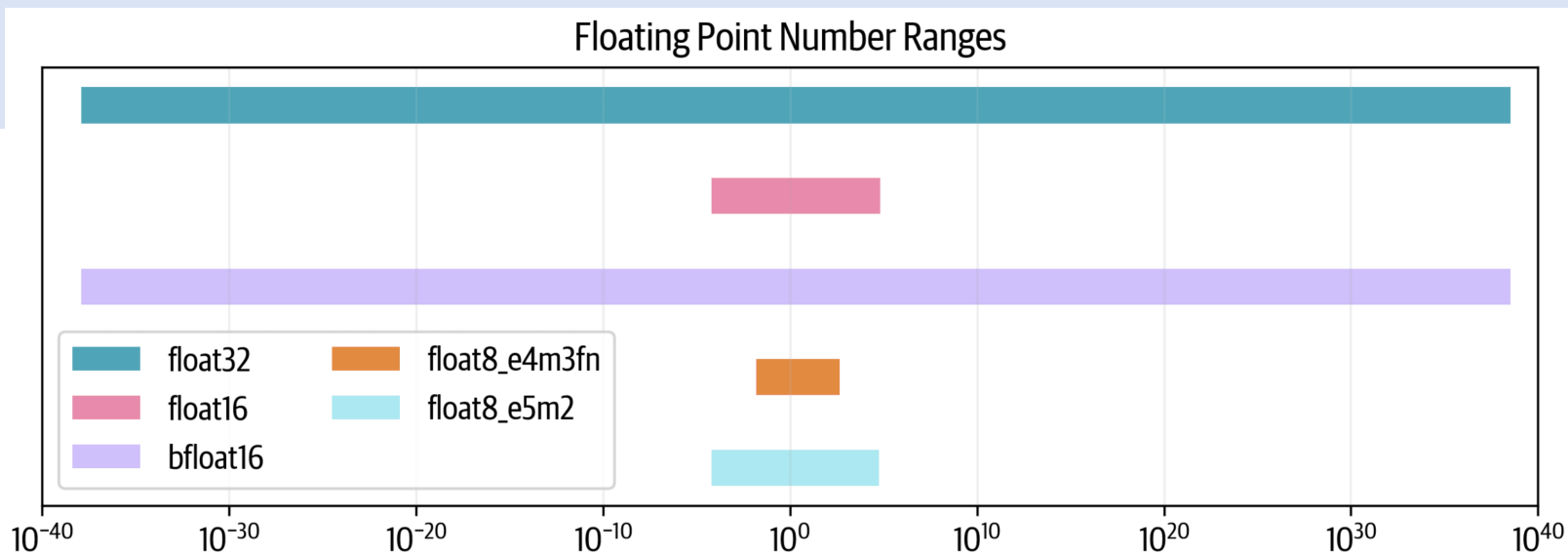
浮点数的原理可以通过回忆数字的科学记数法来简单说明，例如： -5.734×10^7 ，其中首先有符号，然后是尾数和指数。因此，可以用自适应精度表示跨越广泛量级的数字。尽管float32是默认值，但在PyTorch中还有一系列可用的浮点数格式：

Format	Total bits	Sign	Exponent	Mantissa
float32	32	1	8	23
float16	16	1	5	10
bfloat16	16	1	8	7
float8 (e4m3)	8	1	4	3
float8 (e5m2)	8	1	5	2

深入GPU——融合、线程和混合

减少总比特数是有代价的，但可以在如何支付上有一定的控制权

可以在尾数或指数上牺牲比特。因此，也存在两种浮点8（FP8）格式，根据指数和尾数命名，这样可以灵活地选择最合适的格式。让看看每种格式的可能数字范围：

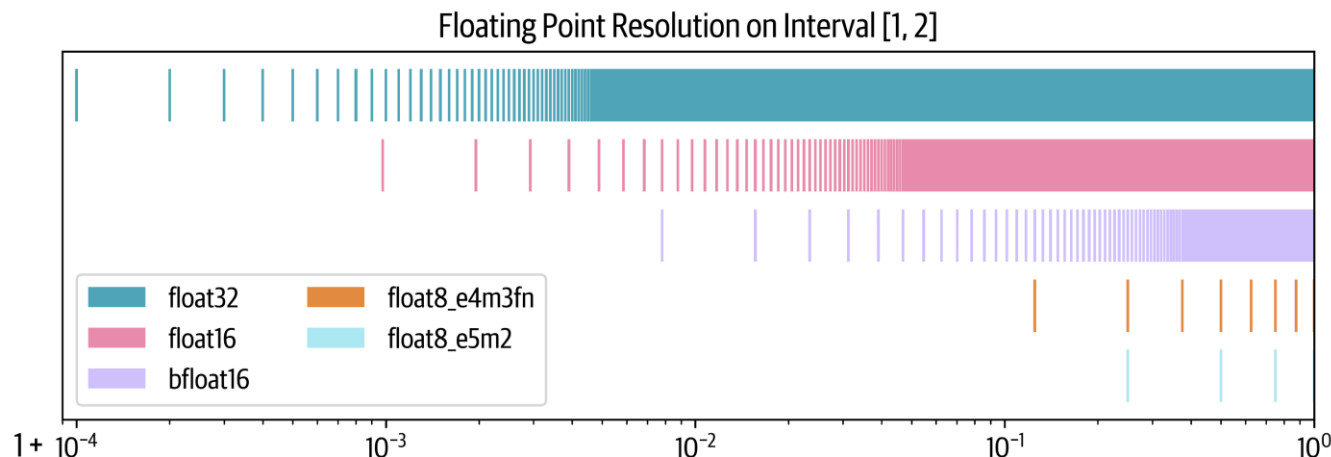


深入GPU——融合、线程和混合

可以看到

float32跨越了80个数量级，而float16牺牲了大量的范围，而bfloat16则保持了完整的范围。两种float8格式进一步减少了范围：e5m2可以保持float16的范围，但e4m3的范围更小。

为什么有些格式能够保持完整的范围，而有些则不能？让通过在1和2之间绘制10,000个点来调查它们的分辨率。每个点将根据每种格式四舍五入到最近的表示数字：



FP16和BF16训练

天真地将所有张量和操作切换到float16很遗憾不起作用，结果通常是发散的损失。然而，原始的混合精度训练论文提出了三个技巧来匹配float32训练：

- **FP32权重的副本**：FP16权重有两个可能的问题。在训练过程中，一些权重可能变得非常小，并被舍入到0。然而，即使权重本身并不接近0，如果更新非常小，那么它们在加法过程中可能会发生下溢。一旦权重变为0，它们将在剩余的训练过程中保持为0，因为没有梯度信号再传递过来。
- **损失缩放**：同样会遇到梯度的问题，因为梯度往往比1小得多，因此存在下溢的风险。一个简单而有效的策略是在反向传播之前缩放损失，并在反向传播之后取消缩放梯度。这确保了反向传播过程中没有下溢，并且缩放不会影响训练，因为取消缩放是在进一步处理梯度之前（例如，裁剪）和优化步骤。
- **累积**：最后，在执行某些16位精度下的算术运算（如平均值或求和）时，也可能面临下溢或上溢。解决方案是在运算过程中将中间结果累积在FP32中，并且只在最终结果转换回16位精度。

深入GPU——融合、线程和混合

FP8 预训练

即使完美地将通信与计算重叠，最终也会遇到硬件本身的底层理论 FLOPS 限制，即硬件上每个单独操作的效率。这就是数值精度变得至关重要的地方。例如，在 NVIDIA 的 H100 GPU 上，FP8 矩阵乘法（GEMM 操作）实现了 BF16 理论 FLOPS 的两倍，使得低精度训练成为进一步优化的有吸引力的途径。

最近的研究——包括 FP8-LM、torchao 和 DeepSeek-V3

已经证明了 FP8 训练对于大规模模型的潜力。然而，FP8 预训练引入了一个重大挑战：稳定性。在较低精度下，数值不稳定性往往导致损失发散，使得难以匹配高精度训练的精度。

知道，对于固定模型大小，不稳定性与学习率的增加成正比，这使得 FP8 预训练特别棘手。

深入GPU——融合、线程和混合

首次成功的大规模FP8混合精度训练在DeepSeek-V3技术报告中公开报道

作者仔细分析了正向传递（Fprop）的每个操作，以及激活（Dgrad）和权重（Wgrad）的反向传递。与BF16混合精度训练类似，一些聚合和主权重保持在更高精度，而操作本身则在FP8中执行。

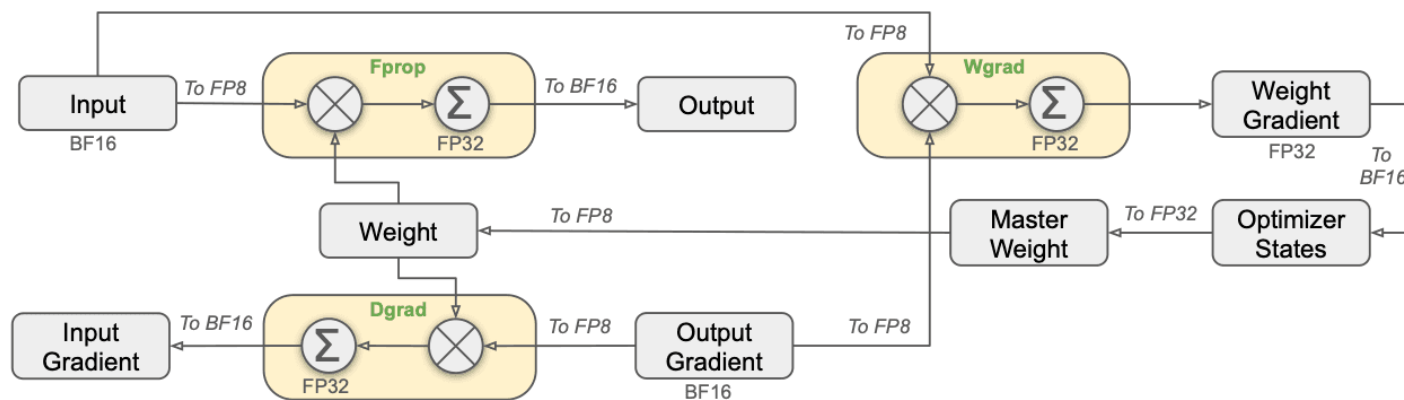


Figure 6 | The overall mixed precision framework with FP8 data format. For clarification, only the Linear operator is illustrated.

深入GPU——融合、线程和混合

为了从高精度（例如，FP32或BF16）切换到范围更小的低精度（例如，FP16或FP8）

需要对激活值的范围进行归一化，例如通过计算它们的绝对最大值。DeepSeek-V3进一步引入了一种特定的量化方案，其中每个瓦片对范围进行归一化：输入/激活为1x128，权重和尺度元素为128x128。这使得归一化对激活中的异常值的影响较小。作者还提出了一些额外的技巧来进一步减少内存和通信的开销，可以在技术报告的第3.3节中了解更多信息。

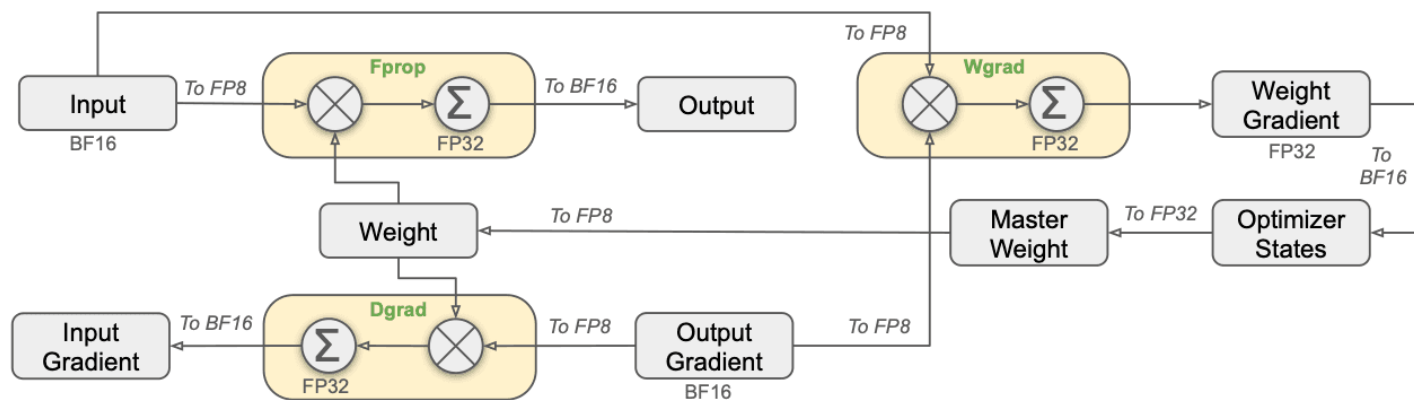


Figure 6 | The overall mixed precision framework with FP8 data format. For clarification, only the Linear operator is illustrated.

深入GPU——融合、线程和混合

FP8训练的几种已知方法的总结:

GEMM's precision	Master model weights	Accumulated gradients	Model weights	Gradients	Optimizer states	Total memory	
BF16 with FP32 mixed precision baseline	BF16	FP32	FP32	BF16	BF16	FP32 + FP32	$4 + 4 + 2 + 2 + 4 + 4 = 20$ bytes
The above without FP32 grad accumulation	BF16	FP32	n/a	BF16	BF16	FP32 + FP32	$4 + 2 + 2 + 4 + 4 = 16$ bytes (20% reduction)
Transformer engine	FP8	n/a	n/a	FP32	FP32	FP32 + FP32	$4 + 4 + 4 + 4 = 16$ bytes (20% reduction)
FP8-LM's O3 level	FP8	FP16	FP16	FP8	FP8	FP8 + FP16	$2 + 2 + 1 + 1 + 1 + 2 = 9$ bytes (55% reduction)
DeepSeek-V3	FP8	FP32	FP32	FP8	BF16	BF16 + BF16	$4 + 4 + 1 + 2 + 2 + 2 = 15$ (25% reduction)
Nanotron's FP8	FP8	BF16	FP32	FP8	FP8	FP8 + FP8	$2 + 4 + 1 + 1 + 1 + 1 = 10$ bytes (50% reduction)